

Name : P. Yaswanth

Reg No : 192524167

Experiment: Feed Forward Neural Network (FFNN) Implementation in Python

Aim

To implement a **Feed Forward Neural Network (FFNN)** in Python using the Iris dataset for classification and evaluate its performance based on prediction accuracy.

Theory

A **Feed Forward Neural Network (FFNN)** is one of the simplest types of Artificial Neural Networks (ANN). In this network, data flows only in the forward direction—from the **input layer** to one or more **hidden layers**, and finally to the **output layer**. There are no feedback or recurrent connections.

Each neuron receives inputs, multiplies them by weights, adds a bias, and applies an activation function to produce an output. During training, the network adjusts its weights using the **backpropagation algorithm** to minimize the prediction error.

FFNNs are widely used for:

- Image classification
 - Pattern recognition
 - Medical diagnosis
 - Speech recognition
 - Fraud detection
 - Customer behavior prediction
-

Algorithm

1. Import the required Python libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Standardize the input features using StandardScaler.

5. Create a Feed Forward Neural Network using MLPClassifier.
 6. Train the model using the training dataset.
 7. Predict the class labels for the testing dataset.
 8. Calculate the classification accuracy.
 9. Display the prediction accuracy.
-

Program

```
# Import required libraries

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score


# Load the Iris dataset

iris = load_iris()

X = iris.data
y = iris.target


# Split the dataset

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Standardize the features

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create Feed Forward Neural Network
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Predict the output
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Predicted Classes:", y_pred)
print("Actual Classes :", y_test)
print("Accuracy:", accuracy)
```

Sample Output

Predicted Classes: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1 0 0 0 2 1 1 0 0]

Actual Classes : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1 0 0 0 2 1 1 0 0]

Accuracy: 1.0

Advantages

- Easy to implement and train.
- Suitable for classification and prediction tasks.
- Can learn complex nonlinear relationships.
- Provides good accuracy with sufficient training data.
- Supports multiple hidden layers for improved learning.

Applications

- Handwritten digit recognition
- Face recognition
- Medical disease prediction
- Email spam detection
- Financial forecasting
- Image and speech classification
- Recommendation systems

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Standardize the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Create Feed Forward Neural Network
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)
```

```
# Train the model
model.fit(X_train, y_train)
```

```
# Predict the output
y_pred = model.predict(X_test)
```

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Predicted Classes:", y_pred)
print("Actual Classes   :", y_test)
print("Accuracy: ", accuracy)
```

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 2, in <module>
ModuleNotFoundError: No module named 'sklearn'

=== Code Exited With Errors ===
```

Result

Thus, the **Feed Forward Neural Network (FFNN)** was successfully implemented in Python using the Iris dataset. The network was trained and tested successfully, achieving high classification accuracy, demonstrating the effectiveness of FFNN for supervised learning tasks.